BIG DATA PROJECT

Réalisé par :

- Marc NGANGNGANG
- Mazigh MOHAMEDI
- Jean brice GBAMI

Msc Computer science and Data Science

1- Data & Service Debrief

Titre du projet : Service de streaming de données financières en temps réel (API Finnhub)

Source : Finnhub Stock API

Domaine : Marchés financiers/ Données boursières et cryptos

Endpoints :
- WebSocket (streaming temps réel) : wss://ws.finnhub.io?token={token}
- Api REST(derniers prix connus) : https://finnhub.io/api/v1/quote

Licence : Finnhub (Gratuit)

1- Data & Service Debrief

Schéma de données - Evènement de trade enrichi :

Le schéma de données correspond à un événement de transaction crypto enrichi. Chaque message combine les données de trade reçues en temps réel via WebSocket avec un snapshot de cotation récupéré périodiquement via l'API REST de Finnhub. Les timestamps d'événement et d'ingestion sont explicitement distingués afin de permettre des analyses temps réel robustes.


```
{
  "source": "string",
  "asset_class": "string",
  "event_type": "string",
  "symbol": "string",
  "exchange": "string | null",
  "pair": "string",
  "price": "float",
  "volume": "float",
  "notional": "float | null",
  "event_ts_ms": "int",
  "ingest_ts_ms": "int",

  "open": "float | null",
  "high": "float | null",
  "low": "float | null",
  "close": "float | null",
  "prev_close": "float | null",
  "quote_event_ts_ms": "int | null",
  "quote_ingest_ts_ms": "int | null"
}
```

1- Data & Service Debrief

Problématique :

Les investisseurs particuliers, analystes financiers et systèmes de trading automatisés ont besoin d'un accès à faible latence aux données de marché en temps réel afin de surveiller les variations de prix, détecter des tendances et réagir rapidement aux événements du marché. Les sources de données existantes sont souvent coûteuses ou complexes à intégrer.

Problème résolu :

Ce service exploite les flux temps réel de Finnhub et fournit une couche de données normalisée, fiable et facilement exploitable pour :

- La surveillance des prix en direct
- La détection de tendances à court terme
- L'alimentation de tableaux de bord en temps réel et d'outils d'analyse

1- Data & Service Debrief

Metriques de marché :

- L’indice de volatilité
- La déviation du prix par rapport à la moyenne
- Le prix actuel du marché
- Evolution du volume par rapport au prix
- variation du prix actuel par rapport au prix d’ouverture
- Variation du volume par rapport à la journée précédente

Utilisateurs cilbes :

- Investisseurs particuliers
- Tradeurs

1- Data & Service Debrief

Structure Gitlab :

```
streaming-Financial-Data/
|
├── README.md
|
├── Finnhub/
|
|     └── finnhub_to_kafka.py
|
├── Streaming/
|
|     └── kafka_to_influx.scala
|
└── Vidéo démo
```

Lien Gitlab : https://gitlab.com/Marc2024/streaming-financial-data

1- Infrastructure Configuration

Cluster map / component :

Ci dessous la liste des différents composants présents dans notre architecture et leur rôle.

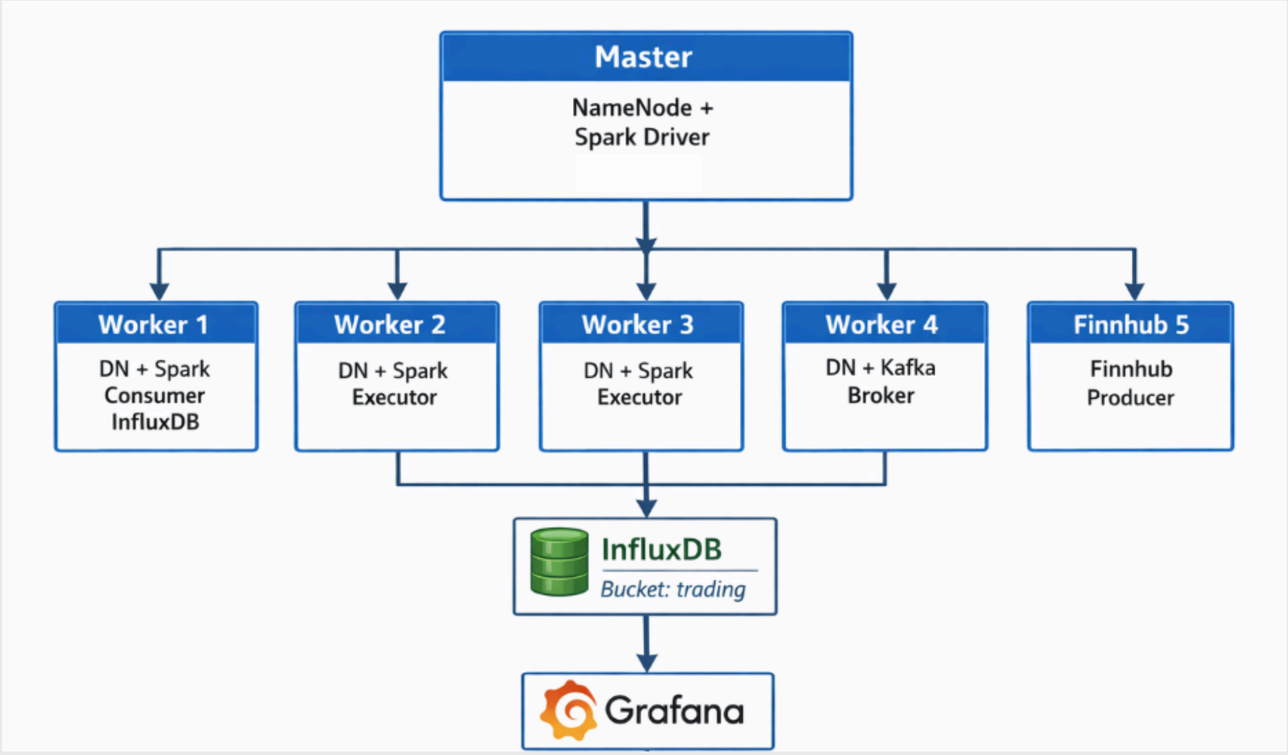
Component	Host + Port	Notes
HDFS NameNode	Master : 9000	Gère le namespace HDFS, coordonne les DataNodes
HDFS DataNode	Worker 1, 2,3,4 et 5	Stocke les blocs HDFS
Spark Driver	Master : 8080	Coordonne les jobs Spark, lit les topics Kafka
Spark Executor	Worker 1, 2 et 3	Exécute les tâches Spark
Kafka Broker	Worker4 : 9092	Reçoit et distribue les messages crypto
Finnhub Producer	Worker 5	Publie les trades sur Kafka topic
InfluxDB	Worker1 : 8086	Stocke les trades dans influxDB
Grafana	Worker2 : 3000	Visualisation des données InfluxDB

IP	Name
10.0.0.134	master
10.0.0.135	worker1
10.0.0.130	worker2
10.0.0.131	worker3
10.0.0.118	worker4
10.0.0.119	worker5

1- Infrastructure Configuration

Diagramme de l'infrastructure :

- Le Master ordonnance les Jobs
- Kafka → Spark : Spark Streaming lit le topic crée .
- Spark → InfluxDB : Les trades sont transformés et écrits dans le bucket InfluxDB.
- InfluxDB → Grafana : Visualisation temps réel des trades crypto.



Conclusion

Ce projet a permis de déployer une architecture Big Data distribuée et orientée streaming, basée sur un cluster composé d'un master et de cinq workers. Elle s'appuie sur Kafka pour l'ingestion temps réel, Spark Streaming pour le traitement, InfluxDB pour le stockage des séries temporelles et Grafana pour la visualisation.

L'architecture assure un flux de données cohérent de bout en bout et constitue une solution évolutive répondant efficacement aux besoins de traitement temps réel, tout en illustrant les principes fondamentaux des systèmes Big Data modernes.

THANK YOU